SENG 31242

System Design Specification (SDS)

Scooter Rent Kandy

By

Jojeevan – SE/2021/013

Kajanan – SE/2021/032

Gobinath – SE/2021/045

Mohamad – SE/2021/048

A report submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science Honours in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Version History

Version	Date	Author(s)	Description
1.0	2026/04/01	Jojeevan, Kajanan, Gobinath, Mohamad	Initial SDS document created
1.1	2026/04/08	Jojeevan, Kajanan, Gobinath, Mohamad	Added sequence diagrams and UI design sections
1.2	2026/04/11	Jojeevan, Kajanan, Gobinath, Mohamad	Finalized security design, deployment, and references
2.0	2026/04/18	Jojeevan, Kajanan, Gobinath, Mohamad	Final version: all diagrams embedded, full TOC, exact SRS styling

Abstract

This System Design Specification (SDS) document describes the complete technical design of the Scooter Rent Kandy system, a web-based scooter rental management platform developed for Kandy Travels and Tours, Kandy, Sri Lanka. The document covers the high-level architectural design, database schema with Entity-Relationship diagram, UML class diagram, sequence diagrams for all major use cases, user interface navigation flow, security design, and deployment network architecture.

The system is designed using a layered MVC architecture with Spring Boot on the backend, React.js on the frontend, and PostgreSQL as the primary data store, all hosted on Amazon Web Services (AWS) in the Asia Pacific (Singapore) region. Key features include real-time GPS tracking via Google Maps API, automated time-and-distance-based billing, digital contract management, camping gear rental integration, role-based access control (RBAC), and an administrative reporting dashboard. All design decisions are derived directly from the approved Software Requirements Specification (SRS).

Keywords: System Design, MVC Architecture, Spring Boot, React.js, PostgreSQL, GPS Tracking, JWT Authentication, AWS Deployment, UML, ER Diagram, RBAC.

Table of Contents

Version History ii
Abstract iii
Table of Contents iv
List of Figures vi
List of Tables vii
Definitions, Acronyms, and Abbreviations viii
Chapter 1: Architectural Design 1
1.1 High-Level Architecture 1
1.2 Justification for Architecture Choice 2
1.3 Scalability and Maintainability Considerations 3
Chapter 2: Database Design 4
2.1 Entity-Relationship (ER) Diagram 4
2.2 Entity Descriptions and Attributes 4
User 4
Scooter 5
Booking 6
Payment 7
GPS Log 7
Camping Gear 8
2.3 Key Relationships 8
2.4 Normalization 9
Chapter 3: Class Diagram 10
3.1 Domain Class Descriptions 10
3.2 Enumerations 11
3.3 Relationships and Multiplicity 12
Chapter 4: Sequence Diagrams 13
4.1 User Registration 13
4.2 Scooter Booking 14
4.3 GPS-Based Billing Calculation 15
4.4 Admin Report Generation 17
Chapter 5: UI/UX Design 19
5.1 Design Principles 19
5.2 Screen Descriptions and Accessibility Considerations 20
5.3 User Navigation Flow 26
Chapter 6: Security Design 27

6.1 Authentication Mechanism 27
6.2 Security Design Summary 27
6.3 Input Validation and Sanitization Strategy 28
6.4 Encryption Standards 29
Chapter 7: Deployment Design 30
7.1 Hosting Plan 30
7.2 Environment Specification 31
7.3 Network Diagram 31
7.4 CI/CD Pipeline Overview 32
Chapter 8: Data Requirements 34
8.1 Overview 34
8.2 Data Entities and Attributes 34
8.2.1 User Data 34
8.2.2 Scooter Data 35
8.2.3 Booking Data 35
8.2.4 Payment Data 36
8.2.5 GPS Log Data 37
8.2.6 Camping Gear Data 37
8.3 Data Volume and Retention 38
8.4 Data Formats and Standards 38
8.5 Data Privacy and Compliance 39
References 40

List of Figures

Figure 1: Scooter Rent Kandy – High-Level System Architecture Diagram 1
Figure 2: Scooter Rent Kandy – Entity-Relationship (ER) Diagram (Chen Notation) 4
Figure 3: Scooter Rent Kandy – UML Class Diagram 10
Figure 4: Sequence Diagram – User Registration 13
Figure 5: Sequence Diagram – Scooter Booking 14
Figure 6: Sequence Diagram – GPS-Based Billing Calculation 16
Figure 7: Sequence Diagram – Admin Report Generation 17
Figure 8: Scooter Rent Kandy – UI Navigation Flow Diagram 26
Figure 9: Scooter Rent Kandy – Deployment Network Diagram 32

List of Tables

Table 1: Version History i
Table 2: Definitions, Acronyms, and Abbreviations vii
Table 3: Architectural Layers and Responsibilities 2
Table 4: User Entity Attributes 5
Table 5: Scooter Entity Attributes 5
Table 6: Booking Entity Attributes 6
Table 7: Payment Entity Attributes 7
Table 8: GPS Log Entity Attributes 8
Table 9: Camping Gear Entity Attributes 8
Table 10: Normalization Analysis (1NF – BCNF) 9
Table 11: Domain Class Descriptions 10
Table 12: Enumerations 11
Table 13: Sequence Steps – User Registration 13
Table 14: Sequence Steps – Scooter Booking 15
Table 15: Sequence Steps – GPS-Based Billing 16
Table 16: Sequence Steps – Admin Report Generation 18
Table 17: Screen Descriptions and Accessibility 20
Table 18: Security Design Summary 27
Table 19: Encryption Standards 29
Table 20: AWS Deployment Components 30
Table 21: Environment Specification 31
Table 22: User Data Entity – Full Attributes 34
Table 23: Scooter Data Entity – Full Attributes 35
Table 24: Booking Data Entity – Full Attributes 35
Table 25: Payment Data Entity – Full Attributes 36
Table 26: GPS Log Data Entity – Full Attributes 37
Table 27: Camping Gear Data Entity – Full Attributes 37
Table 28: Data Volume and Retention Policy 38
Table 29: References 40

Definitions, Acronyms, and Abbreviations

Table 2: Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition / Description
SDS	System Design Specification
SRS	Software Requirements Specification
MVC	Model-View-Controller architectural pattern
API	Application Programming Interface
GPS	Global Positioning System
JWT	JSON Web Token – used for stateless authentication
RBAC	Role-Based Access Control
ER	Entity-Relationship (used in database design diagrams)
TLS	Transport Layer Security
AES	Advanced Encryption Standard
CI/CD	Continuous Integration / Continuous Delivery
AWS	Amazon Web Services
UML	Unified Modelling Language
WCAG	Web Content Accessibility Guidelines
UI/UX	User Interface / User Experience
PCI DSS	Payment Card Industry Data Security Standard
OTP	One-Time Password
UUID	Universally Unique Identifier
KMS	AWS Key Management Service
WAF	Web Application Firewall
CDN	Content Delivery Network
RDS	Relational Database Service – managed AWS PostgreSQL
EC2	Elastic Compute Cloud – AWS virtual server
S3	Simple Storage Service – AWS object storage
ALB	Application Load Balancer

Term / Acronym	Definition / Description
ECR	Elastic Container Registry – AWS Docker image storage

Chapter 1: Architectural Design

This chapter presents the high-level system architecture, justification for the chosen design pattern, and the scalability and maintainability strategies employed in the Scooter Rent Kandy system. All design decisions are directly traceable to the functional and non-functional requirements documented in the approved SRS.

1.1 High-Level Architecture

A layered Model-View-Controller (MVC) architecture has been adopted for the Scooter Rent Kandy system. The architecture is decomposed into six distinct layers, each with well-defined responsibilities. This separation of concerns ensures that changes to one layer do not propagate uncontrollably to others, thereby improving maintainability and testability.

Figure 1: Scooter Rent Kandy – High-Level System Architecture



Figure 1: Scooter Rent Kandy – High-Level System Architecture Diagram

Table 3: Architectural Layers and Responsibilities

Layer	Technology / Component	Responsibility
Presentation	React.js, Google Maps Widget, Stripe.js	Renders the user interface; handles user interactions; communicates with the API over HTTPS.
API Gateway	AWS ALB, Spring Security, TLS 1.3	Terminates SSL; authenticates JWT tokens; enforces RBAC; routes requests to business logic.
Business Logic	Spring Boot Services (BookingService, GPSBillingService, PaymentService, ReportService)	Implements all business rules; orchestrates calls to the data layer and external services.
Data Access	Spring Data JPA, Hibernate ORM, HikariCP	Abstracts database queries using the Repository pattern; manages connection pooling.
Database	Amazon RDS PostgreSQL 16 (Multi-AZ)	Persists all system data; provides ACID transactions and automated backups.
External Services	Google Maps API, Stripe, AWS SES/SNS, AWS S3	GPS distance calculation, payment processing, email/SMS notifications, and file storage.

1.2 Justification for Architecture Choice

The layered MVC pattern has been selected over alternatives such as microservices and monolithic architectures for the following reasons. First, the project scope and team size (four developers) are better suited to a layered monolith, which reduces deployment and operational complexity. Second, MVC clearly separates user interface concerns from business logic and data access, enabling parallel development streams. Third, the architecture can be migrated towards microservices incrementally if the system scales beyond its initial scope. A cloud-native deployment on AWS has been adopted to provide elastic scalability, managed database services, and integrated monitoring without the overhead of on-premises infrastructure management.

1.3 Scalability and Maintainability Considerations

- Horizontal scaling is achieved through an AWS Auto Scaling Group attached to the Application Load Balancer; additional EC2 instances launch automatically when CPU utilization exceeds 70%.
- The stateless REST API design ensures that any backend instance can serve any request, enabling true horizontal scalability without sticky sessions.
- Database connection pooling via HikariCP is configured to a maximum of 20 connections per instance, preventing connection exhaustion under load.
- Modular Spring Boot service classes follow the Single Responsibility Principle, allowing individual services (e.g., GPSBillingService) to be refactored or extracted without affecting others.
- RESTful API versioning (/api/v1/) enables future breaking changes to be introduced without disrupting existing clients.
- Docker containerization ensures environment parity between development, staging, and production.

Chapter 2: Database Design

This chapter presents the Entity-Relationship (ER) diagram in Chen notation, entity descriptions with full attribute tables, key relationships, and normalization analysis for the Scooter Rent Kandy database schema. The schema is implemented in PostgreSQL 16 and accessed via Spring Data JPA (Hibernate). All entity primary keys use UUID type to ensure globally unique, non-sequential identifiers.

2.1 Entity-Relationship (ER) Diagram

The ER diagram below uses Chen notation: rectangles for strong entities, a double rectangle for the weak entity (Booking Gear), ellipses for attributes, underlined ellipses for primary keys, and diamonds for relationships. Cardinality labels (1 and N) are placed on the connecting lines.

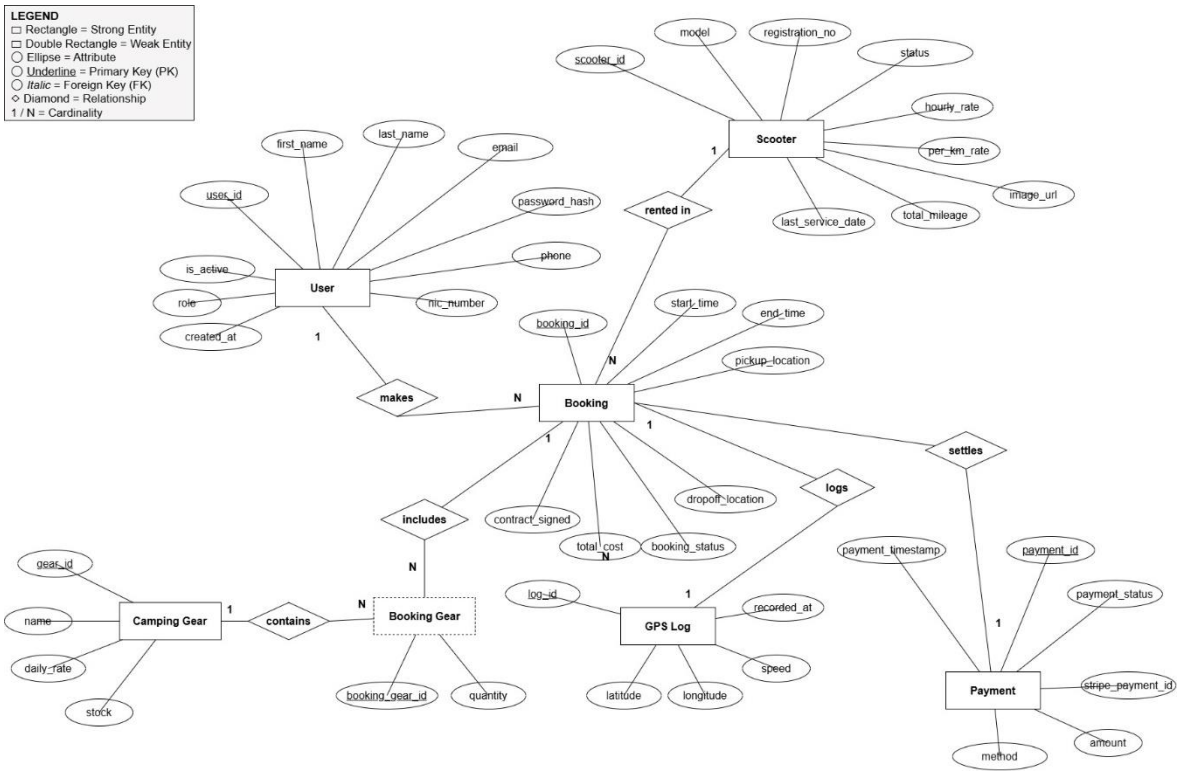


Figure 2: Scooter Rent Kandy – Entity-Relationship (ER) Diagram (Chen Notation)

2.2 Entity Descriptions and Attributes

User

The User entity stores information about all registered users of the system, including both customers and administrators.

Table 4: User Entity Attributes

Attribute	Data Type	Constraint	Description
userId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
firstName	VARCHAR(100)	NOT NULL	User's given name
lastName	VARCHAR(100)	NOT NULL	User's family name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login credential; must be in valid email format
passwordHash	VARCHAR(255)	NOT NULL	BCrypt hashed password (cost factor 12); plain text never stored
phone	VARCHAR(20)	NOT NULL	Mobile contact in international format (+94XXXXXXXXX)
nicNumber	VARCHAR(20)	NOT NULL, UNIQUE	Sri Lanka NIC number; encrypted at rest using AES-256
role	ENUM	NOT NULL, DEFAULT=USER	USER | ADMIN | SUPER_ADMIN – determines RBAC access level
isActive	BOOLEAN	DEFAULT TRUE	TRUE = active account; FALSE = blocked by administrator
createdAt	TIMESTAMP	NOT NULL	Account creation timestamp in ISO 8601 UTC format

Scooter

The Scooter entity stores details of every scooter in the rental fleet managed by the administrator.

Table 5: Scooter Entity Attributes

Attribute	Data Type	Constraint	Description
scooterId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key

Attribute	Data Type	Constraint	Description
model	VARCHAR(150)	NOT NULL	Scooter model name (e.g., Honda Dio, Yamaha Ray-ZR)
registrationNo	VARCHAR(30)	NOT NULL, UNIQUE	Official Sri Lanka vehicle registration plate number
status	ENUM	NOT NULL	AVAILABLE | RENTED | MAINTENANCE – real-time fleet status
hourlyRate	DECIMAL(10,2)	NOT NULL, > 0	Rental charge per hour in Sri Lankan Rupees (LKR)
perKmRate	DECIMAL(10,2)	NOT NULL, >= 0	Distance-based charge per kilometer in LKR
imageUrl	VARCHAR(500)	NULLABLE	AWS S3 URL for the scooter image (JPEG/PNG, max 5 MB)
totalMileage	DECIMAL(10,2)	DEFAULT 0	Cumulative distance travelled by the scooter in kilometers
lastServiceDate	DATE	NULLABLE	Date of most recent maintenance service (ISO 8601)

Booking

The Booking entity records every rental transaction initiated by a user, from reservation through to completion or cancellation.

Table 6: Booking Entity Attributes

Attribute	Data Type	Constraint	Description
bookingId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
userId (FK)	UUID	NOT NULL	References User.userId
scooterId (FK)	UUID	NOT NULL	References Scooter.scooterId
startTime	TIMESTAMP	NOT NULL	Rental start date and time (ISO 8601 UTC)
endTime	TIMESTAMP	NULLABLE	Rental end date/time; NULL if rental is still active
pickupLocation	VARCHAR(300)	NOT NULL	GPS address string of the scooter pickup point

Attribute	Data Type	Constraint	Description
dropLocation	VARCHAR(300)	NULLABLE	GPS address of return point; NULL until rental ends
status	ENUM	NOT NULL	PENDING | ACTIVE | COMPLETED | CANCELLED
totalCost	DECIMAL(10,2)	NULLABLE	Calculated final cost in LKR; NULL until booking completes
contractSigned	BOOLEAN	DEFAULT FALSE	TRUE = digital rental contract accepted by the user

Payment

The Payment entity records the financial transaction associated with each confirmed booking.

Table 7: Payment Entity Attributes

Attribute	Data Type	Constraint	Description
paymentId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
bookingId (FK)	UUID	NOT NULL, UNIQUE	References Booking.bookingId (one payment per booking)
stripePaymentId	VARCHAR(255)	NULLABLE	Stripe PaymentIntent reference ID
amount	DECIMAL(10,2)	NOT NULL, > 0	Total amount charged to the user in LKR
method	ENUM	NOT NULL	CARD | ONLINE – payment method used at checkout
status	ENUM	NOT NULL	PENDING | SUCCESS | FAILED | REFUNDED
paidAt	TIMESTAMP	NULLABLE	Date/time payment was successfully processed (ISO 8601 UTC)

GPS Log

The GPS Log entity records real-time location data transmitted by the scooter GPS device during an active rental, used for distance calculation and speed violation monitoring.

Table 8: GPS Log Entity Attributes

Attribute	Data Type	Constraint	Description
logId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
bookingId (FK)	UUID	NOT NULL	References Booking.bookingId for the active rental
latitude	DECIMAL(9,6)	NOT NULL	GPS latitude coordinate in WGS84 decimal degrees
longitude	DECIMAL(9,6)	NOT NULL	GPS longitude coordinate in WGS84 decimal degrees
speed	DECIMAL(5,2)	NOT NULL, >= 0	Speed of the scooter in km/h at the time of logging
recordedAt	TIMESTAMP	NOT NULL	ISO 8601 UTC; logged every 30 seconds by the GPS device

Camping Gear

The Camping Gear entity stores details of optional camping equipment that users may add to a scooter booking.

Table 9: Camping Gear Entity Attributes

Attribute	Data Type	Constraint	Description
gearId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
name	VARCHAR(200)	NOT NULL	Gear name (e.g., Camping Tent, Gas Cooker, LED Light)
dailyRate	DECIMAL(10,2)	NOT NULL, > 0	Daily rental charge per item in LKR
stock	INTEGER	NOT NULL, >= 0	Number of units currently available in inventory

2.3 Key Relationships

- User → Booking (1 to N): A user may place zero or many bookings. Each booking belongs to exactly one user.

- Scooter → Booking (1 to N): A scooter may appear in zero or many bookings sequentially. Only one booking may be ACTIVE per scooter at any time (enforced by a partial unique index on scooterId WHERE status = ACTIVE).
- Booking → Payment (1 to 1): Each confirmed booking has exactly one associated payment record (enforced by UNIQUE constraint on Payment.bookingId).
- Booking → GPS Log (1 to N): An active booking generates many GPS log entries, recorded every 30 seconds by the scooter device.
- Booking ↔ Camping Gear (M to N): A booking may include zero or many camping gear items; represented by a Booking Gear junction (weak) entity with a quantity attribute.

2.4 Normalization

The database schema has been designed to comply with Third Normal Form (3NF). The table below summaries how each normal form has been addressed.

Table 10: Normalization Analysis (1NF – BCNF)

Normal Form	Requirement	Application to Scooter Rent Kandy
1NF	Atomic values; no repeating groups	All attributes store single atomic values. Camping gear items stored in a separate CampingGear entity referenced via the BookingGear junction table.
2NF	Full functional dependency on PK	All non-key attributes are fully functionally dependent on their single-column UUID primary key. No partial dependencies exist.
3NF	No transitive dependencies	Billing rates stored on Scooter, not duplicated in Booking. Payment method is an ENUM on Payment entity, eliminating transitive dependency.
BCNF	Every determinant is a candidate key	Scooter availability computed from Booking records; no derived attributes stored that could violate BCNF.

A controlled denormalization is applied to the total Cost column of the Booking entity. Although this value can be computed from GPS log data and rates, it is persisted after calculation for immutable billing record and audit purposes. This is a documented exception to 3NF.

Chapter 3: Class Diagram

The UML Class Diagram below models the core domain classes of the Scooter Rent Kandy system. Each class corresponds directly to a JPA entity or service class in the Spring Boot application. The diagram presents all attributes with types and visibility modifiers (- private, + public), all method signatures with parameters and return types, and all relationships with multiplicity labels. The colour scheme matches the project draw.io standard.

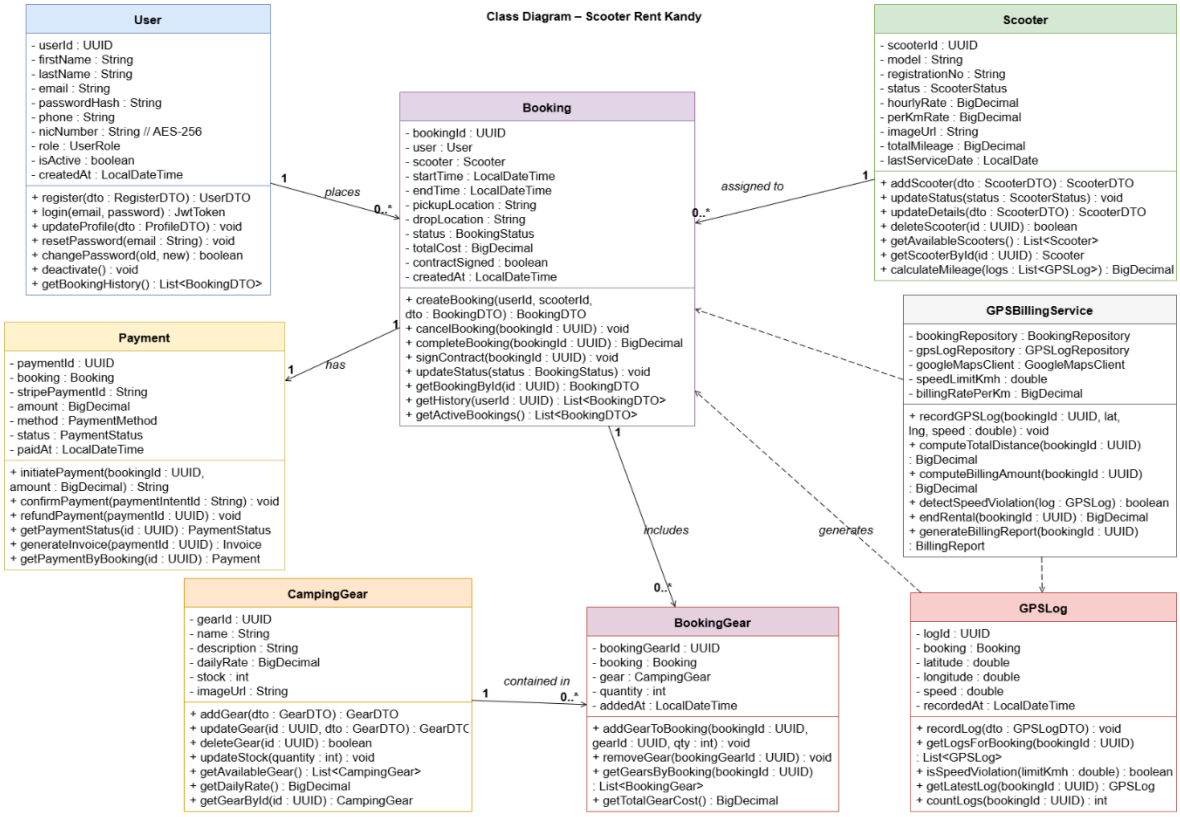


Figure 3: Scooter Rent Kandy – UML Class Diagram

3.1 Domain Class Descriptions

Table 11: Domain Class Descriptions

Class	Responsibility	Key Relationships
User	Represents a registered customer or administrator. Manages authentication, profile, and booking history.	1..N Booking (places)
Scooter	Represents a rental vehicle. Tracks availability, pricing rates, and cumulative mileage.	1..N Booking (assigned to)

Class	Responsibility	Key Relationships
Booking	Central entity capturing the full rental lifecycle from PENDING through ACTIVE to COMPLETED or CANCELLED.	References User and Scooter; has 1 Payment, N GPSLog, N BookingGear
Payment	Records the financial transaction for a booking. References Stripe PaymentIntent; card data never stored locally.	1..1 Booking
CampingGear	Represents optional rental equipment available as booking add-ons.	M..N Booking via BookingGear junction
BookingGear	Junction entity implementing M:N between Booking and CampingGear, with a quantity attribute.	N..1 Booking, N..1 CampingGear
GPSBillingService	Processes GPS logs, computes distance via Google Maps Distance Matrix API, calculates final rental bill.	<<uses>> Booking and GPSLog
GPSLog	Stores real-time positional data emitted by the GPS device every 30 seconds during an active rental.	N..1 Booking

3.2 Enumerations

Table 12: Enumerations

Enumeration	Values	Usage
UserRole	USER, ADMIN, SUPER_ADMIN	Assigned to User to control RBAC permissions at method level.
ScooterStatus	AVAILABLE, RENTED, MAINTENANCE	Controls scooter visibility in browse results and booking eligibility.
BookingStatus	PENDING, ACTIVE, COMPLETED, CANCELLED	Tracks the lifecycle state of every booking record.
PaymentMethod	CARD, ONLINE	Records the payment method used at checkout.
PaymentStatus	PENDING, SUCCESS, FAILED, REFUNDED	Tracks the state of a payment from initiation to settlement.

3.3 Relationships and Multiplicity

- User (1) ———— (0..*) Booking : A user may have zero or many bookings over their account lifetime.
- Scooter (1) ———— (0..*) Booking : A scooter appears in zero or many bookings sequentially over its fleet lifetime.
- Booking (1) ———— (1) Payment : Every completed booking has exactly one payment record.
- Booking (1) ———— (0..*) GPS Log : An active booking generates many GPS log entries every 30 seconds.
- Booking (0..*) ─M:N─ (0..*) CampingGear : Represented by the BookingGear junction class with a quantity attribute.
- GPSBillingService ──<<uses>>── Booking and GPSLog : Dependency (dashed arrow); the service reads but does not own these entities.

Chapter 4: Sequence Diagrams

Sequence diagrams are provided for all four major use cases. Each diagram follows UML 2.5 notation: solid arrows represent synchronous calls, dashed arrows represent return messages, and activation bars indicate periods of component execution.

4.1 User Registration

This diagram illustrates the complete registration flow from form submission through client-side validation, server-side DTO validation, BCrypt password hashing, AES-256 NIC encryption, database insertion, and welcome email dispatch via AWS SES.

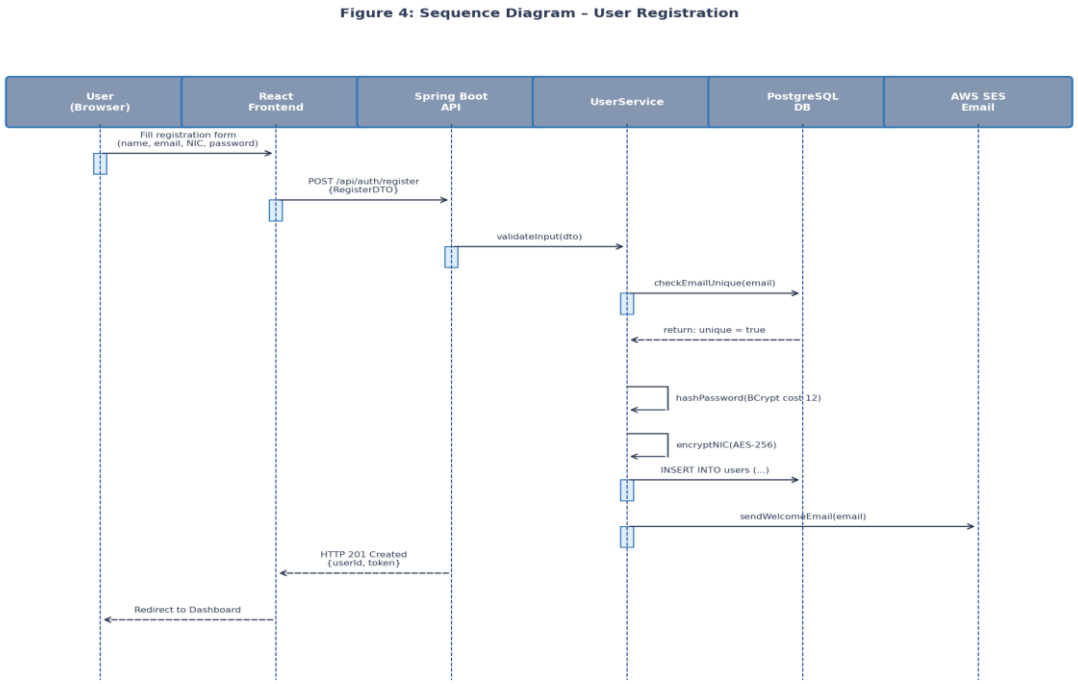


Figure 4: Sequence Diagram – User Registration

Table 13: Sequence Steps – User Registration

#	From	To	Message / Action
1	User (Browser)	React Frontend	Fill registration form and submit
2	React Frontend	Spring Boot API /auth/register	POST {name, email, phone, NIC, password}
3	Spring Boot API	UserService	validateInput(dto)
4	UserService	PostgreSQL DB	checkEmailUnique(email)

#	From	To	Message / Action
5	PostgreSQL DB	UserService	return: unique = true
6	UserService	–	hashPassword(BCrypt cost 12) [self-call]
7	UserService	–	encryptNIC(AES-256) [self-call]
8	UserService	PostgreSQL DB	INSERT INTO users (...)
9	UserService	AWS SES	sendWelcomeEmail(email)
10	Spring Boot API	React Frontend	HTTP 201 Created {userId, token}
11	React Frontend	User (Browser)	Redirect to Dashboard

4.2 Scooter Booking

This diagram covers the complete booking flow: scooter availability validation, booking record creation, Stripe PaymentIntent creation, webhook confirmation upon payment success, booking activation, and SMS confirmation via AWS SNS.

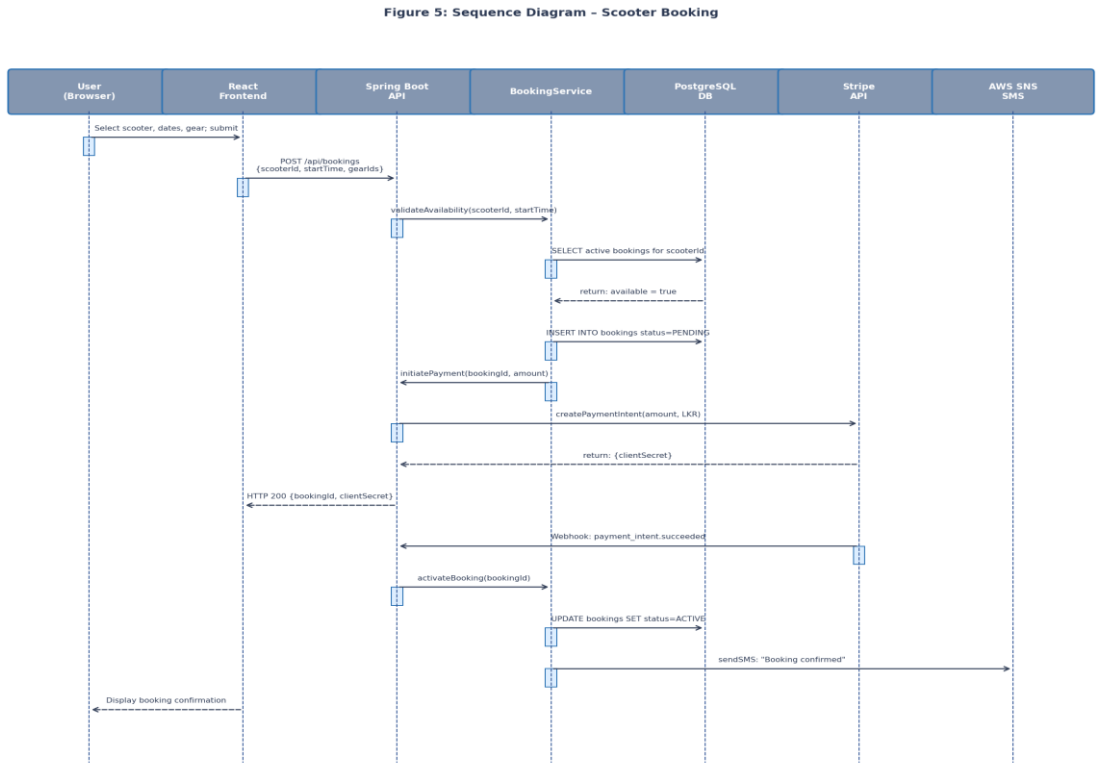


Figure 5: Sequence Diagram – Scooter Booking

Table 14: Sequence Steps – Scooter Booking

#	From	To	Message / Action
1	User (Browser)	React Frontend	Select scooter, dates, gear; submit booking
2	React Frontend	Spring Boot API /bookings	POST {scooterId, startTime, gearIds}
3	Spring Boot API	BookingService	validateAvailability(scooterId, startTime)
4	BookingService	PostgreSQL DB	SELECT active bookings for scooterId
5	PostgreSQL DB	BookingService	return: available = true
6	BookingService	PostgreSQL DB	INSERT INTO bookings (...) status=PENDING
7	BookingService	PaymentService	initiatePayment(bookingId, amount)
8	PaymentService	Stripe API	createPaymentIntent(amount, currency=LKR)
9	Stripe API	PaymentService	return: clientSecret
10	Spring Boot API	React Frontend	HTTP 200 {bookingId, clientSecret}
11	React Frontend	Stripe.js	confirmCardPayment(clientSecret)
12	Stripe	Spring Boot Webhook /webhooks/stripe	payment_intent.succeeded event
13	Spring Boot Webhook	BookingService	activateBooking(bookingId)
14	BookingService	PostgreSQL DB	UPDATE bookings SET status=ACTIVE
15	BookingService	AWS SNS	sendSMS(userId, "Booking confirmed")
16	React Frontend	User (Browser)	Display booking confirmation

4.3 GPS-Based Billing Calculation

This diagram illustrates real-time GPS data collection (POST every 30 seconds), speed violation detection, end-rental trigger, total distance computation via Google Maps Distance Matrix API, billing calculation, and invoice generation.

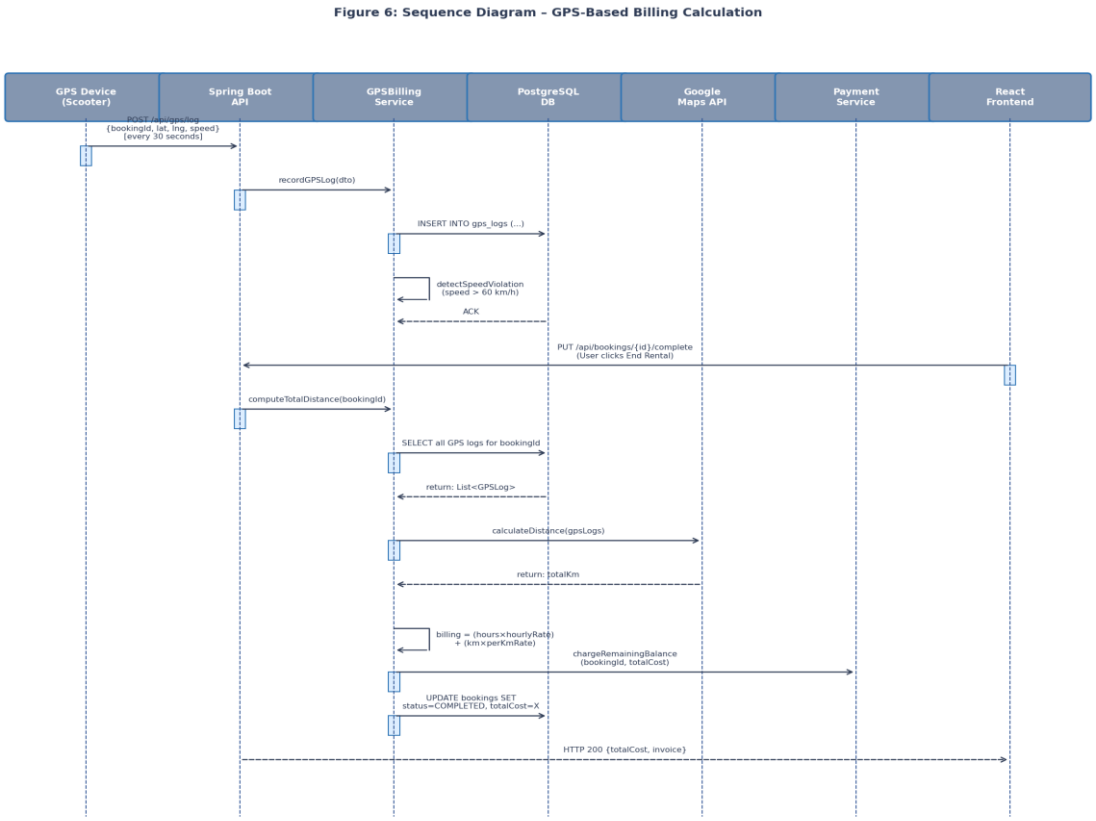


Figure 6: Sequence Diagram – GPS-Based Billing Calculation

Table 15: Sequence Steps – GPS-Based Billing

#	From	To	Message / Action
1	Scooter GPS Device	Spring Boot API /gps/log	POST {bookingId, lat, lng, speed, timestamp} [every 30s]
2	Spring Boot API	GPSBillingService	recordGPSLog(dto)
3	GPSBillingService	PostgreSQL DB	INSERT INTO gps_logs (...)
4	GPSBillingService	–	detectSpeedViolation(speed, 60 km/h) [self-call]
5	User	React Frontend	Click "End Rental"
6	React Frontend	Spring Boot API /bookings/{id}/complete	PUT request
7	Spring Boot API	GPSBillingService	computeTotalDistance(bookingId)
8	GPSBillingService	Google Maps API	calculateDistance(gpsLogs)
9	Google Maps API	GPSBillingService	return: totalKm
10	GPSBillingService	–	billing = (hours × hourlyRate) + (km × perKmRate)

#	From	To	Message / Action
11	GPSBillingService	PaymentService	chargeRemainingBalance(bookingId, totalCost)
12	BookingService	PostgreSQL DB	UPDATE bookings SET status=COMPLETED, totalCost=X
13	Spring Boot API	React Frontend	HTTP 200 {totalCost, invoice}
14	React Frontend	User (Browser)	Display invoice and receipt

4.4 Admin Report Generation

This diagram covers admin report generation: JWT role validation, data aggregation from PostgreSQL, report DTO construction, optional PDF/CSV export with upload to AWS S3, and download URL delivery.

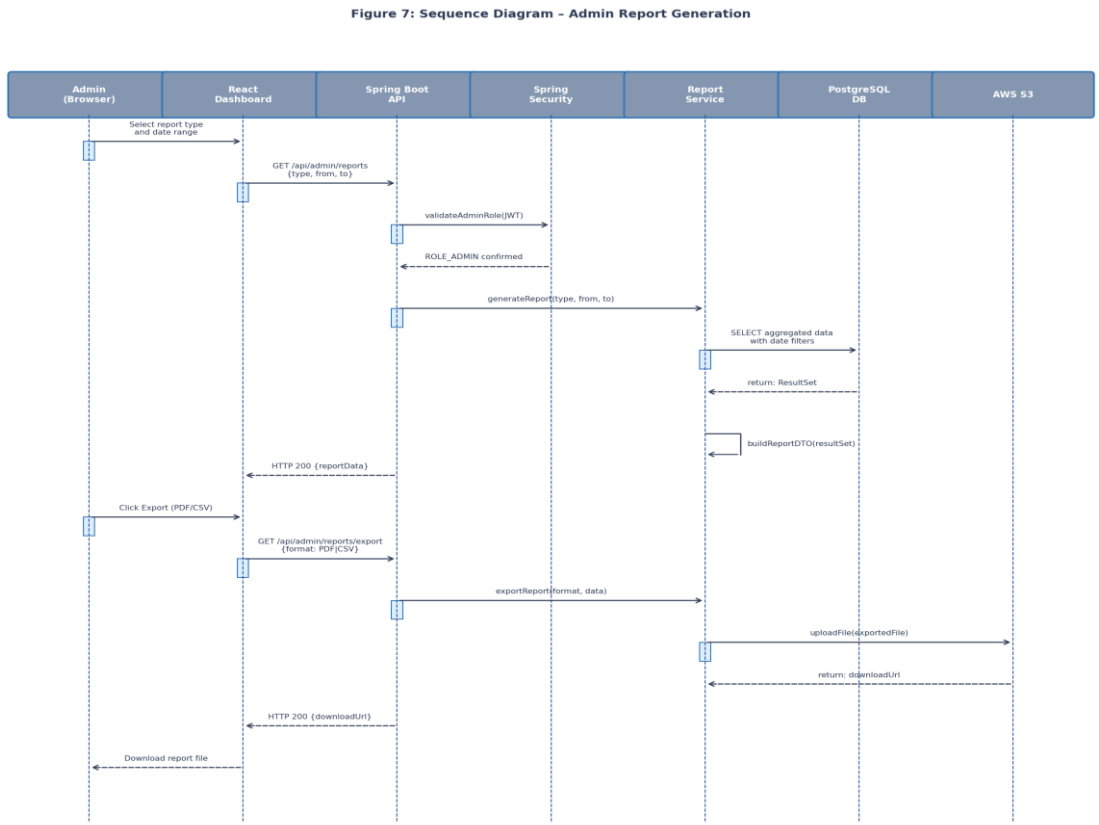


Figure 7: Sequence Diagram – Admin Report Generation

Table 16: Sequence Steps – Admin Report Generation

#	From	To	Message / Action
1	Admin (Browser)	React Admin Dashboard	Select report type and date range
2	React Admin Dashboard	Spring Boot API /admin/reports	GET {type, from, to}
3	Spring Boot API	Spring Security	validateAdminRole(JWT)
4	Spring Security	Spring Boot API	return: ROLE_ADMIN confirmed
5	Spring Boot API	ReportService	generateReport(type, from, to)
6	ReportService	PostgreSQL DB	SELECT aggregated data with date filters
7	PostgreSQL DB	ReportService	return: ResultSet
8	ReportService	–	buildReportDTO(resultSet) [self-call]
9	Spring Boot API	React Admin Dashboard	HTTP 200 {reportData: [...]}
10	Admin (Browser)	Admin	View report or click Export
11	React Admin Dashboard	Spring Boot API /admin/reports/export	GET {format: PDF|CSV}
12	Spring Boot API	ReportService	exportReport(format, data)
13	ReportService	AWS S3	uploadFile(exportedFile)
14	Spring Boot API	React Admin Dashboard	HTTP 200 {downloadUrl}
15	React Admin Dashboard	Admin (Browser)	Download report file

Chapter 5: UI/UX Design

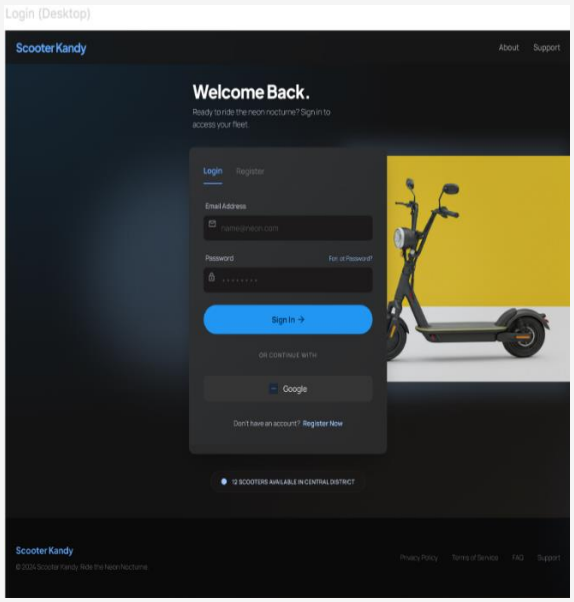
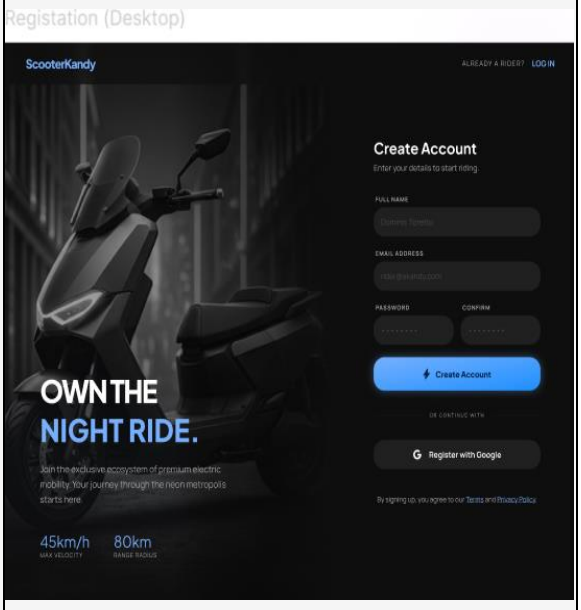
This chapter describes the user interface design principles, key screen descriptions, accessibility considerations, and the navigation flow for the Scooter Rent Kandy system. All screens are implemented using React.js with a responsive CSS Grid and Flexbox layout adapting to desktop (1280px+), tablet (768px), and mobile (375px) viewports.

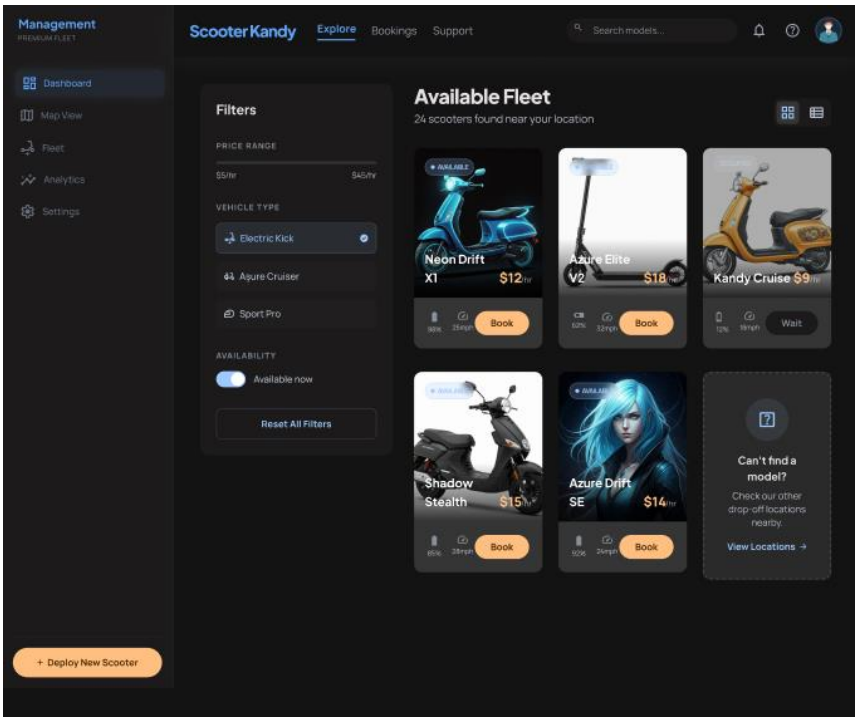
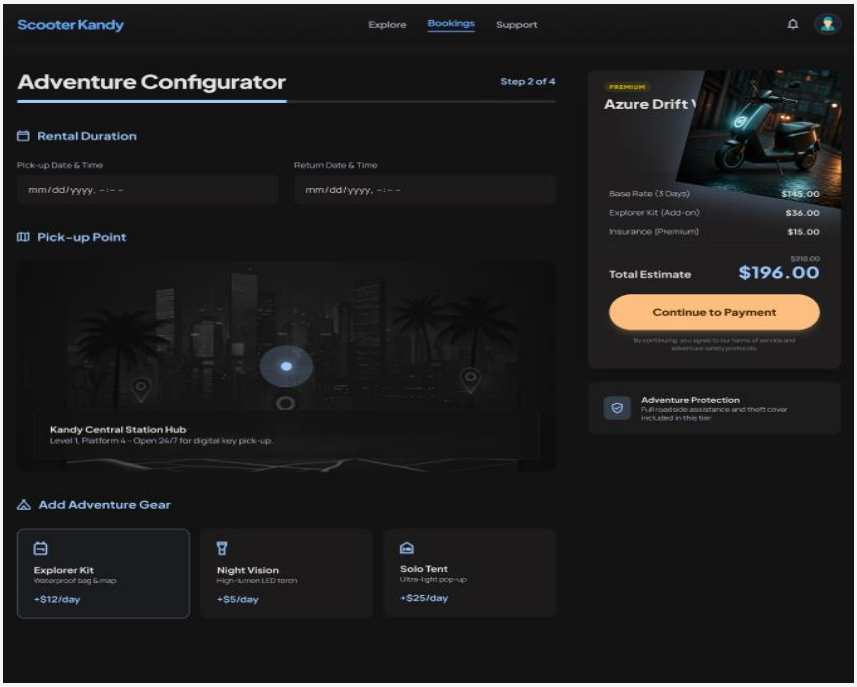
5.1 Design Principles

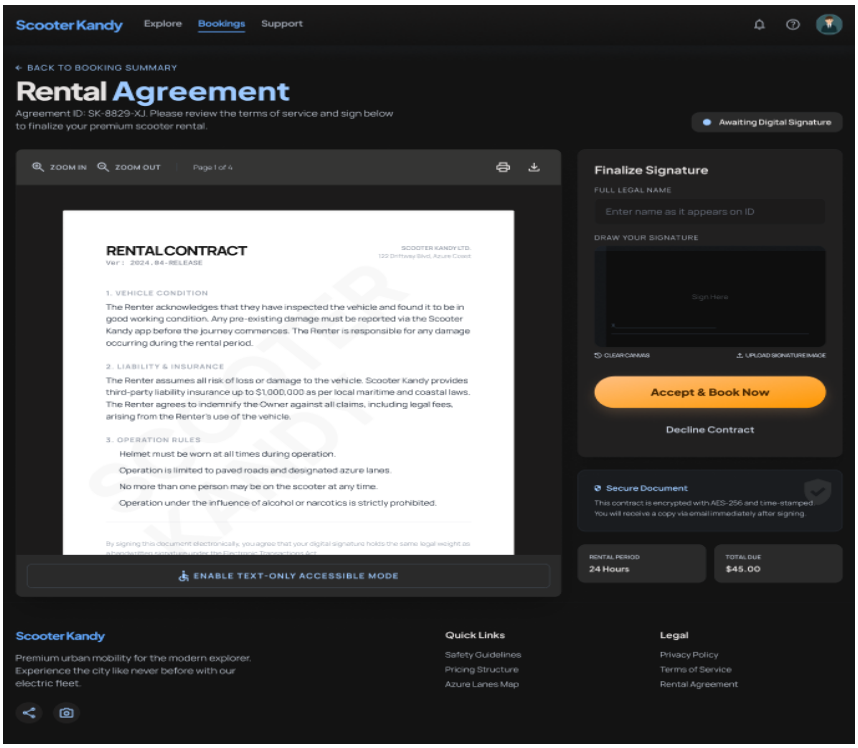
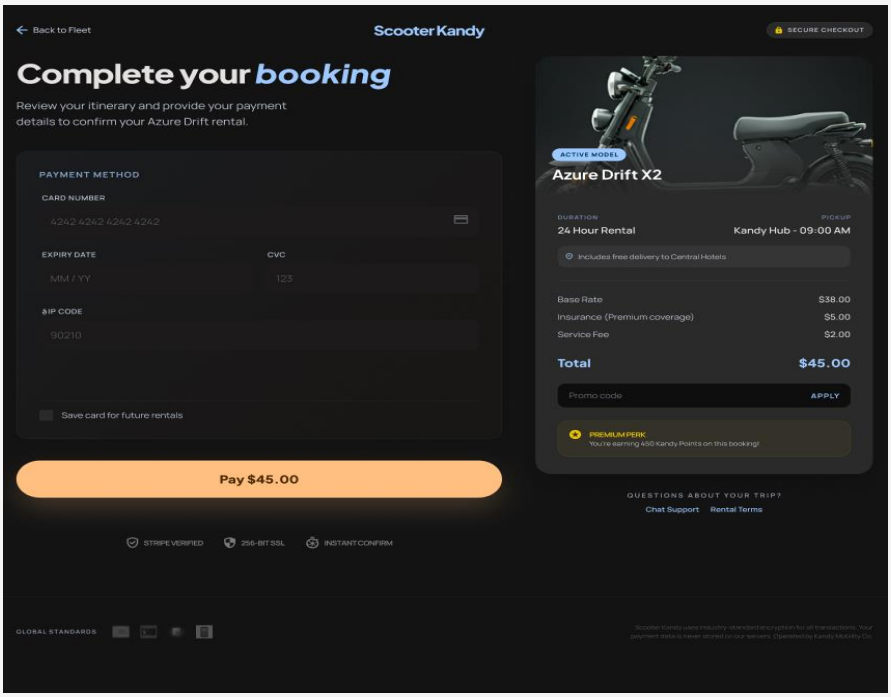
- Consistency: Common components (navigation bars, buttons, form fields) are reused across all screens with identical styling derived from the brand identity (gold #B8860B and dark background with white accents).
- Feedback: Every user action receives immediate visual feedback including loading spinners, success toast notifications, and error banners to indicate system state clearly.
- Simplicity: Each screen focuses on a single primary task; secondary actions are placed in overflow menus or secondary panels to reduce cognitive load.
- Accessibility: All screens target WCAG 2.1 Level AA compliance, as detailed in the table below.
- Responsiveness: The interface adapts to desktop (1280px+), tablet (768px), and mobile (375px) viewports via CSS media queries.

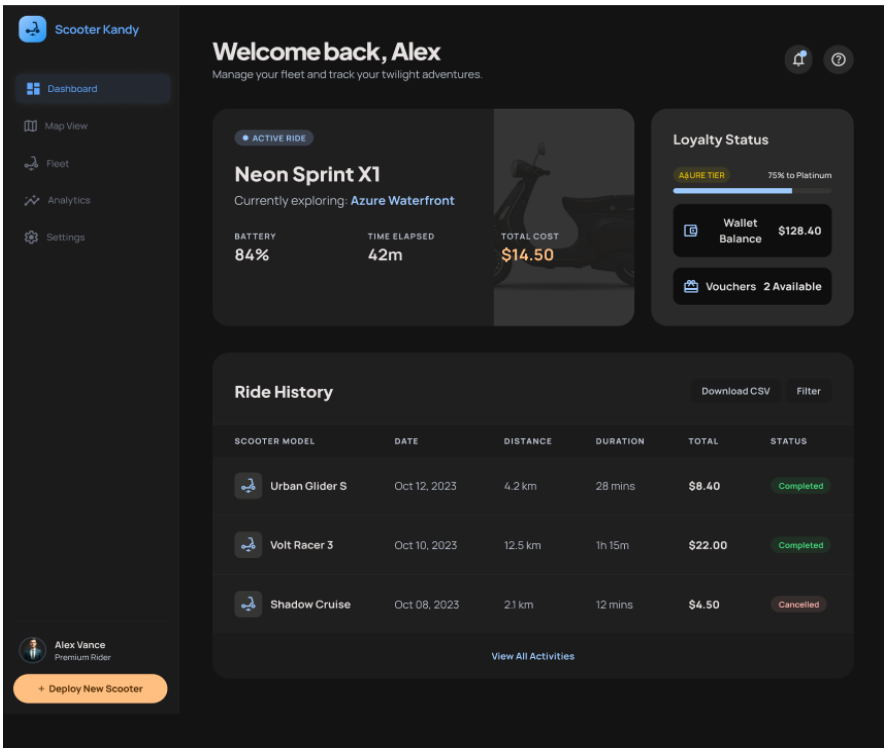
5.2 Screen Descriptions and Accessibility Considerations

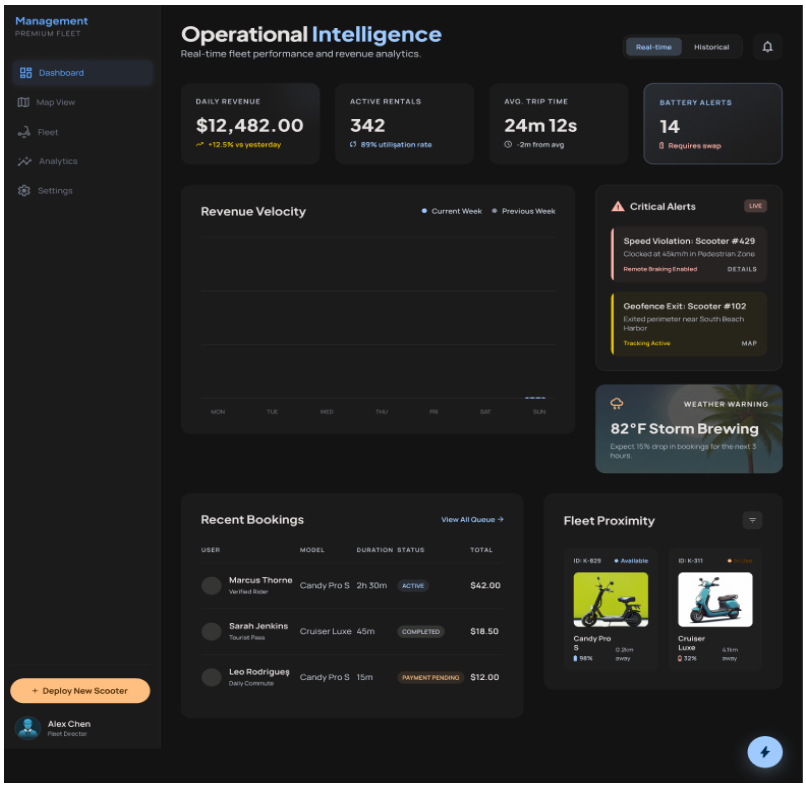
Table 17: Screen Descriptions and Accessibility

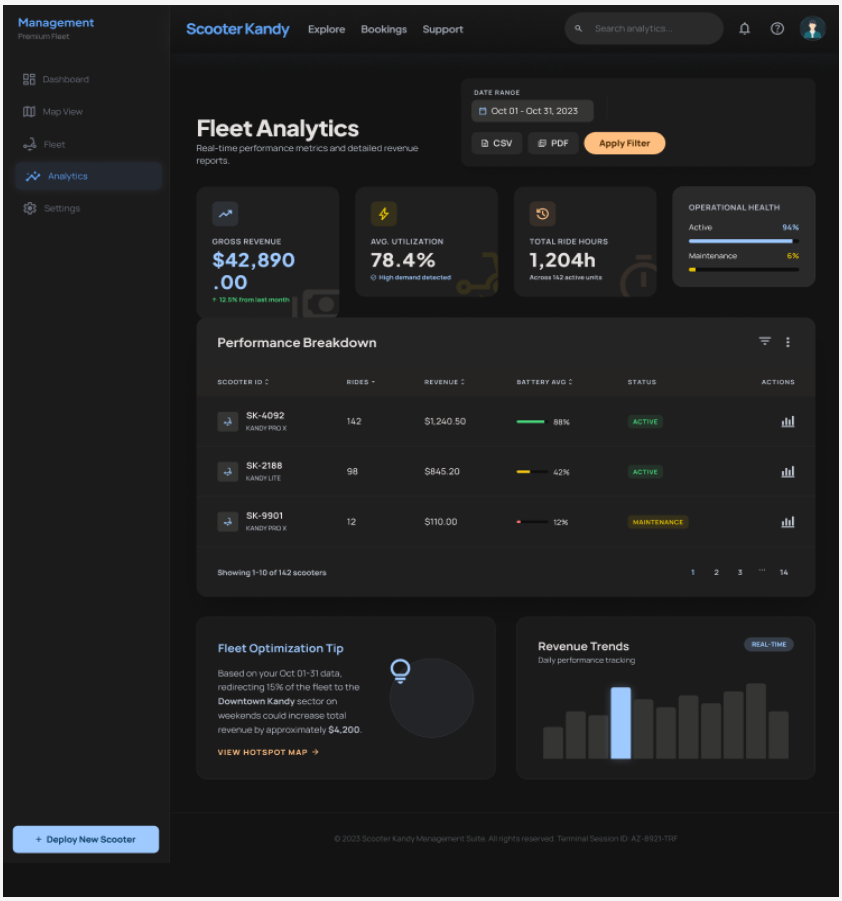
Screen / Module	Key UI Elements	Accessibility Considerations
Registration / Login	Email input, password field, OTP field, forgot-password link	ARIA labels on all inputs; error messages via aria-live; color contrast ratio ≥ 4.5:1 (WCAG 2.1 AA)
		
Scooter Browse & Search	Grid of scooter cards with image, model, price/hr, availability badge; filter sidebar	Keyboard-navigable filter panel; descriptive alt text on scooter images; visible focus indicators

Screen / Module	Key UI Elements	Accessibility Considerations
		
Booking Wizard	Date-time picker, camping gear checklist, Google Maps pickup selector, booking summary	Step progress bar with ARIA progress role; form fields grouped with <fieldset> and <legend>
		
Digital Contract	Inline PDF viewer, signature canvas, accept/decline buttons	PDF viewer with zoom controls; text-based fallback for motor-impaired users

Screen / Module	Key UI Elements	Accessibility Considerations
		
Payment Page	Stripe embedded card element, order summary, secure payment badge	Stripe Elements are fully accessible; inline error messages below each field
		
User Dashboard	Active bookings list, booking history table, notification bell, profile settings	Tables with proper <th> scope; notification count exposed via aria-label

Screen / Module	Key UI Elements	Accessibility Considerations
		
GPS Tracking View	Google Maps with real-time scooter marker, distance counter, speed indicator	Text-based status panel for non-visual users; speed indicator uses color + numeric value
		
Admin Dashboard	Revenue chart, fleet status grid, booking queue, speed violation alerts	Charts with data table alternative; alerts use aria-live=polite announcements

Screen / Module	Key UI Elements	Accessibility Considerations
		
Admin Reports	Date range filter, export buttons, sortable data tables	All export buttons keyboard-accessible; tables sortable with ARIA sort attributes

Screen / Module	Key UI Elements	Accessibility Considerations
		

5.3 User Navigation Flow

The navigation flow diagram below illustrates the complete journeys for both User and Admin roles. Role-based routing is enforced at the React Router level using JWT claim verification. Unauthenticated users are redirected to /login; unauthorized access attempts return HTTP 403 Forbidden.

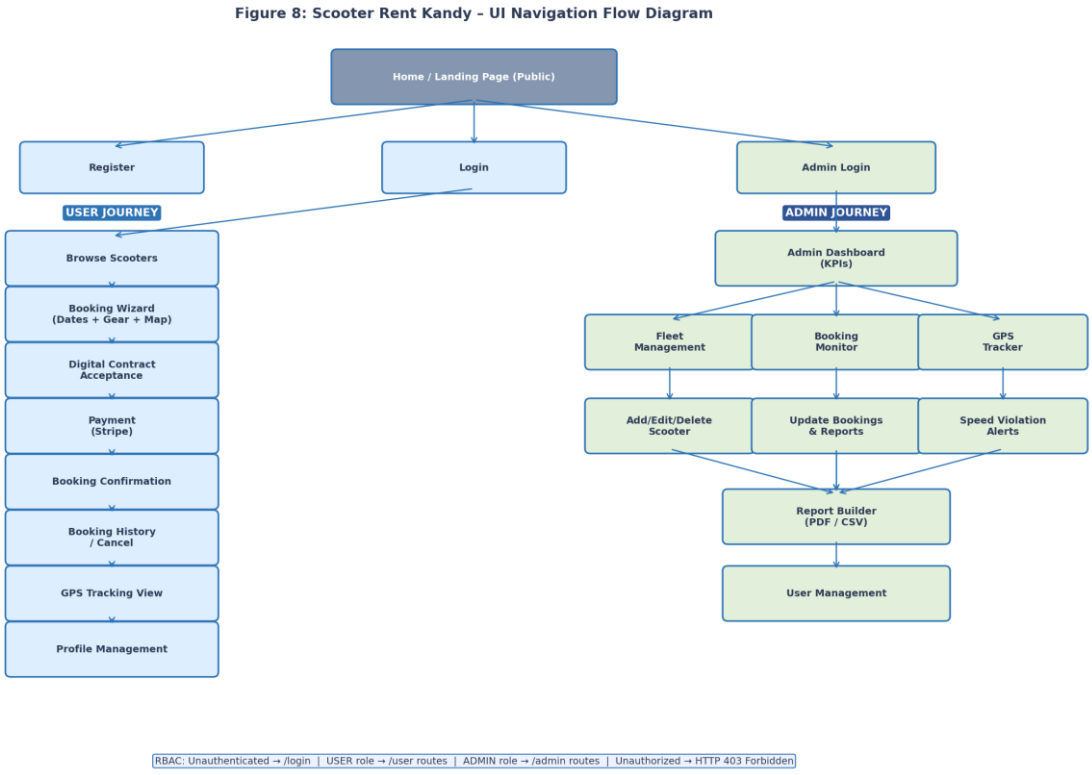


Figure 8: Scooter Rent Kandy – UI Navigation Flow Diagram

An unauthenticated user lands on the Home page and may proceed to Registration or Login. Upon successful login, a USER role is directed to the User Dashboard, from which Scooter Browse, Booking Wizard, Booking History, GPS Tracking View, and Profile screens are accessible. An ADMIN role is directed to the Admin Dashboard, from which Fleet Management, Booking Monitor, GPS Tracker, Report Builder, and User Management are accessible. A logout action from any screen returns the user to the Home page.

Chapter 6: Security Design

The Scooter Rent Kandy system implements a defense-in-depth security strategy with independent protection layers: Network Security (AWS WAF + ALB), Transport Security (TLS 1.3), Application Security (JWT + Spring Security + RBAC), Data Security (AES-256 + BCrypt), and Audit/Monitoring (Spring AOP + AWS CloudWatch). A breach of one layer does not compromise the entire system.

6.1 Authentication Mechanism

Authentication is implemented using JSON Web Tokens (JWT) issued by Spring Security upon successful credential verification. The access token has a validity period of one (1) hour. A refresh token, stored in an HttpOnly, SameSite=Strict cookie to prevent JavaScript access, is valid for seven (7) days and is used to obtain a new access token transparently without requiring the user to re-login.

- Multi-factor authentication via OTP delivered by AWS SNS SMS is required for all ADMIN and SUPER_ADMIN logins, providing an additional protection layer for privileged accounts.
- Failed login attempts are rate-limited to 5 per minute per IP address to prevent brute-force attacks.
- The private RSA key used for JWT signing (RS256 algorithm) is stored in AWS Secrets Manager and is never embedded in application code or environment files.

6.2 Security Design Summary

Table 18: Security Design Summary

Security Aspect	Mechanism	Details
Authentication	JWT (RS256) + Spring Security	Access tokens expire after 1 hour; refresh tokens stored in HttpOnly cookie (7-day expiry).
Authorization	RBAC – Spring Security @PreAuthorize	Roles: USER, ADMIN, SUPER_ADMIN. Method-level annotations enforce access control.
Password Storage	BCrypt (cost factor 12)	Passwords never stored in plain text; BCrypt hashing applied on registration and password reset.

Security Aspect	Mechanism	Details
Data in Transit	TLS 1.3 (HTTPS enforced)	All endpoints served over HTTPS. HTTP → HTTPS permanent redirect (301) at the load balancer.
Data at Rest	AES-256 encryption	Sensitive fields (NIC numbers, Stripe tokens) encrypted at application layer; RDS volume encrypted via AWS KMS.
Input Validation	Server-side + Client-side	Jakarta Bean Validation (JSR-380) on all DTOs; regex whitelisting for GPS coordinates, NIC, phone.
SQL Injection	Parameterized queries via JPA	No raw SQL string concatenation; all queries use prepared statements via Spring Data JPA/Hibernate.
XSS Prevention	Content Security Policy headers	CSP headers restrict inline scripts; React escapes all dynamic content by default.
Payment Security	PCI DSS via Stripe tokenization	Card data never touches our servers; Stripe PaymentIntent API used for all transactions.
Audit Logging	Spring AOP + AWS CloudWatch	All admin actions, login events, and payment events logged with timestamps, IP, and user IDs.

6.3 Input Validation and Sanitization Strategy

All input data is validated at two independent levels. At the client side, React form libraries enforce field-level rules (required fields, email format, numeric ranges for GPS coordinates, password strength). At the server side, Jakarta Bean Validation annotations applied to DTOs enforce constraints declaratively. Input exceeding defined length limits is rejected with HTTP 400 Bad Request. All string inputs are stripped of leading and trailing whitespace before processing. File uploads (scooter images) are restricted to JPEG/PNG formats with a maximum size of 5 MB, validated at both the React frontend and the Spring Boot controller layer.

6.4 Encryption Standards

Table 19: Encryption Standards

Data Category	Standard	Implementation
Data in Transit	TLS 1.3	Enforced via AWS ALB; HTTP permanently redirected to HTTPS (301); HSTS header enabled
Passwords at Rest	BCrypt (cost=12)	Applied in Spring Security PasswordEncoder before any database write
Sensitive PII at Rest	AES-256 GCM	Application-layer encryption via AWS KMS-managed keys; defense in depth over RDS encryption
Database Volume	AES-256	AWS RDS storage encryption using AWS KMS-managed keys
JWT Signature	RS256 (RSA-SHA256)	Private key in AWS Secrets Manager; public key distributed to API instances
S3 Objects	AES-256 SSE-S3	Server-side encryption applied automatically to all S3 objects

Chapter 7: Deployment Design

This chapter describes the AWS hosting plan, environment specifications, deployment network architecture, and CI/CD pipeline. All resources are hosted on Amazon Web Services in the Asia Pacific (Singapore) ap-southeast-1 region to minimize latency for the primary user base in Sri Lanka.

7.1 Hosting Plan

Table 20: AWS Deployment Components

Component	AWS Service	Specification
Frontend	AWS S3 + CloudFront	React build artefacts hosted on S3; served globally via CloudFront CDN with TLS 1.3 at edge.
Backend API	AWS EC2 (t3.medium)	Spring Boot JAR deployed on EC2 within a private VPC subnet; Auto Scaling Group (min 1, max 4 instances).
Database	Amazon RDS PostgreSQL 16	Multi-AZ deployment for high availability; automated daily snapshots retained for 7 days.
File Storage	Amazon S3	Scooter images, rental contracts, and report exports stored with SSE-S3 AES-256 encryption.
Email / SMS	AWS SES + AWS SNS	SES for booking confirmation emails; SNS for SMS OTPs and speed violation alerts.
Load Balancer	AWS Application Load Balancer	Distributes traffic across EC2 instances; terminates TLS; routes /api/* to backend, /* to frontend.
Security	AWS WAF	Filters common web attack patterns (XSS, SQL injection, CSRF) before traffic reaches the ALB.
Monitoring	AWS CloudWatch	CPU, memory, and error-rate alarms; log groups for application and access logs; audit trail.
Container Registry	AWS ECR	Docker images stored and versioned; images pulled by EC2 instances during deployment.
Secrets	AWS Secrets Manager + KMS	JWT RSA keys, database credentials, and third-party API keys managed securely.

7.2 Environment Specification

Table 21: Environment Specification

Attribute	Development	Staging	Production
Infrastructure	Local machine / Docker Compose	AWS EC2 t3.small (single instance)	AWS EC2 t3.medium (Auto Scaling 1–4)
Database	Local PostgreSQL 16	RDS PostgreSQL 16 (single-AZ)	RDS PostgreSQL 16 (Multi-AZ)
Branch	feature/* branches	develop branch	main branch
Logging	Full stack traces enabled	INFO-level logging	ERROR level only
HTTPS	Self-signed certificate	ACM certificate (staging domain)	ACM certificate (production domain)
External APIs	Mock / test mode	Stripe test keys; Twilio test SID	Live production API keys

7.3 Network Diagram

The network architecture places all backend resources within an Amazon VPC. EC2 instances and the RDS cluster reside in private subnets with no direct internet access. The Application Load Balancer is deployed in public subnets and is the only entry point for external traffic. NAT Gateways in each Availability Zone allow backend instances to make outbound API calls without being publicly reachable.

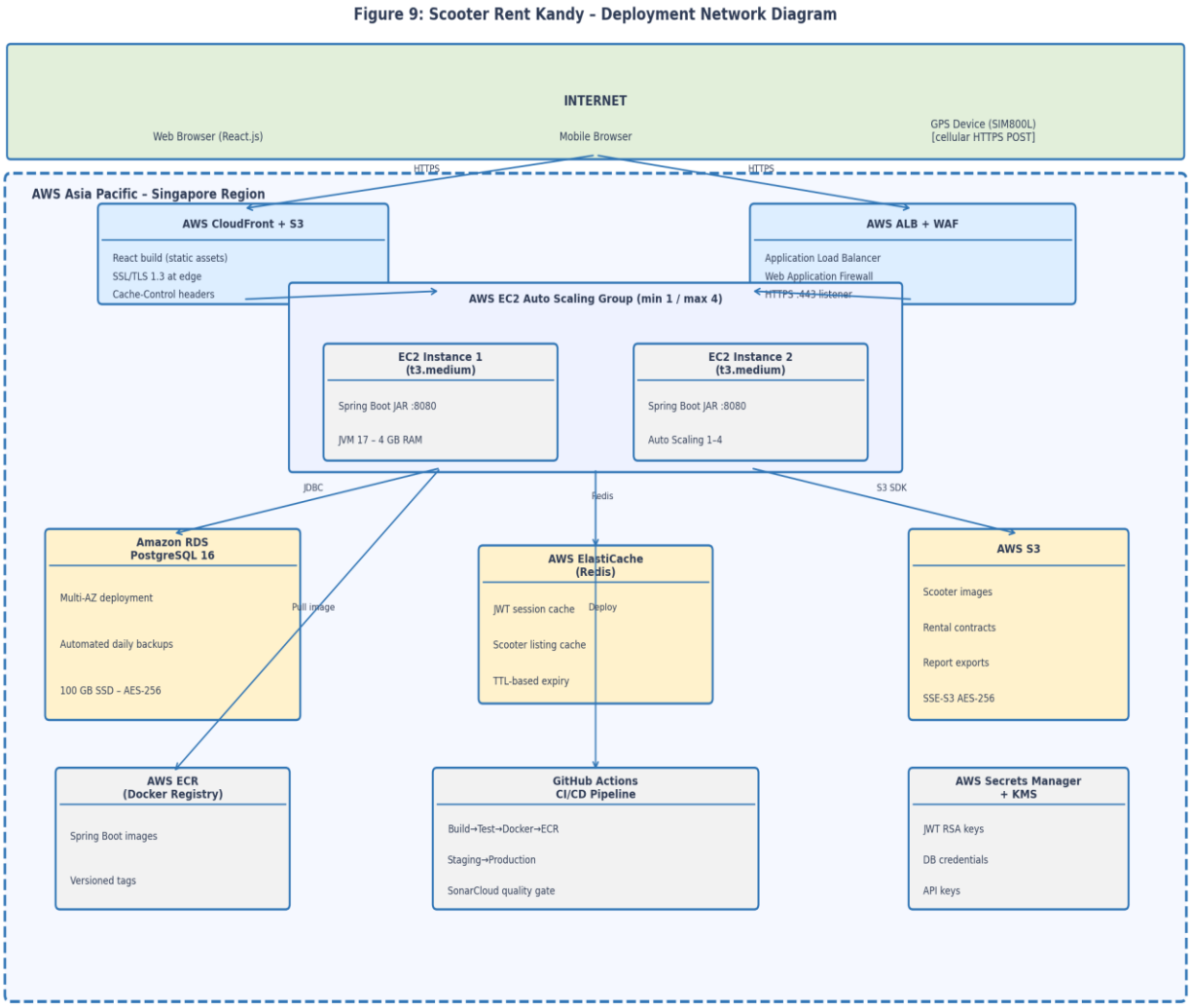


Figure 9: Scooter Rent Kandy – Deployment Network Diagram

7.4 CI/CD Pipeline Overview

A GitHub Actions pipeline is employed for continuous integration and delivery, triggered on every push to the develop and main branches.

- Stage 1 – Build & Test: Maven builds the Spring Boot JAR; JUnit 5 unit and integration tests execute; React components tested with Jest. Minimum 80% code coverage enforced.
- Stage 2 – Code Quality: SonarCloud static analysis checks for code smells, duplication, and security vulnerabilities. The build fails if critical issues are detected.
- Stage 3 – Docker Build: A multi-stage Docker image is built from the Spring Boot JAR and pushed to Amazon ECR with a version tag.

- Stage 4 – Deploy to Staging: The new image is deployed to the staging EC2 instance; automated smoke tests execute against the staging URL.
- Stage 5 – Deploy to Production: On merge to main, a manual approval gate is required before deployment. A rolling update strategy ensures zero downtime.

Chapter 8: Data Requirements

8.1 Overview

This chapter specifies the data requirements of the Scooter Rent Kandy system. Each data entity, its attributes, data types, constraints, example values, and data formats are described. These requirements are derived directly from the functional requirements and database design presented in Chapter 2.

8.2 Data Entities and Attributes

8.2.1 User Data

Table 22: User Data Entity – Full Attributes

Attribute	Data Type	Constraint	Format / Example	Notes
userId	UUID	PK, NOT NULL	550e8400-e29b-...	System-generated key
firstName	VARCHAR(100)	NOT NULL	Jojeevan	User given name
lastName	VARCHAR(100)	NOT NULL	Arumugam	User family name
email	VARCHAR(255)	NOT NULL, UNIQUE	user@example.com	Login credential
passwordHash	VARCHAR(255)	NOT NULL	$2a$12$…	BCrypt hash; never plain text
phone	VARCHAR(20)	NOT NULL	+94771234567	E.164 international format
nicNumber	VARCHAR(20)	NOT NULL, UNIQUE	200012345678V	AES-256 encrypted at rest
role	ENUM	NOT NULL	USER | ADMIN	RBAC access level
isActive	BOOLEAN	DEFAULT TRUE	true / false	FALSE = account blocked
createdAt	TIMESTAMP	NOT NULL	2026-04-01T10:30:00Z	ISO 8601 UTC

8.2.2 Scooter Data

Table 23: Scooter Data Entity – Full Attributes

Attribute	Data Type	Constraint	Format / Example	Notes
scooterId	UUID	PK, NOT NULL	uuid-string	System-generated
model	VARCHAR(150)	NOT NULL	Honda Dio	Scooter model name
registrationNo	VARCHAR(30)	NOT NULL, UNIQUE	WP CAA 1234	Sri Lanka vehicle plate
status	ENUM	NOT NULL	AVAILABLE	Real-time fleet status
hourlyRate	DECIMAL(10,2)	NOT NULL, > 0	250.00	LKR per hour
perKmRate	DECIMAL(10,2)	NOT NULL, >= 0	15.00	LKR per km
imageUrl	VARCHAR(500)	NULLABLE	https://s3…./img.png	AWS S3 public URL
totalMileage	DECIMAL(10,2)	DEFAULT 0	12500.50	Cumulative km
lastServiceDate	DATE	NULLABLE	2026-03-15	ISO 8601 YYYY-MM-DD

8.2.3 Booking Data

Table 24: Booking Data Entity – Full Attributes

Attribute	Data Type	Constraint	Format / Example	Notes
bookingId	UUID	PK, NOT NULL	uuid-string	System-generated
userId	UUID	FK → User, NOT NULL	uuid-string	References User entity

Attribute	Data Type	Constraint	Format / Example	Notes
scooterId	UUID	FK → Scooter, NOT NULL	uuid-string	References Scooter entity
startTime	TIMESTAMP	NOT NULL	2026-04-10T09:00:00Z	ISO 8601 UTC
endTime	TIMESTAMP	NULLABLE	2026-04-10T12:00:00Z	NULL if rental is ongoing
pickupLocation	VARCHAR(300)	NOT NULL	Kandy Lake, Kandy, Sri Lanka	GPS address string
dropLocation	VARCHAR(300)	NULLABLE	Peradeniya Road, Kandy	NULL until rental ends
status	ENUM	NOT NULL	ACTIVE	Booking lifecycle state
totalCost	DECIMAL(10,2)	NULLABLE	875.00	Calculated on completion in LKR
contractSigned	BOOLEAN	DEFAULT FALSE	true / false	Must be TRUE to activate

8.2.4 Payment Data

Table 25: Payment Data Entity – Full Attributes

Attribute	Data Type	Constraint	Format / Example	Notes
paymentId	UUID	PK, NOT NULL	uuid-string	System-generated
bookingId	UUID	FK → Booking, UNIQUE	uuid-string	One payment per booking
stripePaymentId	VARCHAR(255)	NULLABLE	pi_3NxYZ…	Stripe transaction reference
amount	DECIMAL(10,2)	NOT NULL, > 0	875.00	Total amount in LKR

Attribute	Data Type	Constraint	Format / Example	Notes
method	ENUM	NOT NULL	CARD	Payment method used
status	ENUM	NOT NULL	SUCCESS	Payment lifecycle state
paidAt	TIMESTAMP	NULLABLE	2026-04-10T09:05:00Z	NULL until payment succeeds

8.2.5 GPS Log Data

Table 26: GPS Log Data Entity – Full Attributes

Attribute	Data Type	Constraint	Format / Example	Notes
logId	UUID	PK, NOT NULL	uuid-string	System-generated
bookingId	UUID	FK → Booking, NOT NULL	uuid-string	Links log to active booking
latitude	DECIMAL(9,6)	NOT NULL	7.290572	WGS84 decimal degrees
longitude	DECIMAL(9,6)	NOT NULL	80.633728	WGS84 decimal degrees
speed	DECIMAL(5,2)	NOT NULL, >= 0	45.20	Speed in km/h at log time
recordedAt	TIMESTAMP	NOT NULL	2026-04-10T09:15:00Z	ISO 8601 UTC; every 30 s

8.2.6 Camping Gear Data

Table 27: Camping Gear Data Entity – Full Attributes

Attribute	Data Type	Constraint	Format / Example	Notes
gearId	UUID	PK, NOT NULL	uuid-string	System-generated

Attribute	Data Type	Constraint	Format / Example	Notes
name	VARCHAR(200)	NOT NULL	Camping Tent	Gear item name
dailyRate	DECIMAL(10,2)	NOT NULL, > 0	500.00	LKR per day
stock	INTEGER	NOT NULL, >= 0	10	Available quantity

8.3 Data Volume and Retention

Table 28: Data Volume and Retention Policy

Entity	Est. Records (Year 1)	Growth Rate	Retention Policy
User	~500 registered users	~200 per year	Retained indefinitely; deactivated accounts archived after 2 years
Scooter	~20–30 fleet units	~5 per year	Retained permanently; soft-deleted on disposal
Booking	~5,000 bookings	~4,000 per year	Active records kept 1 year; older records archived automatically
Payment	~5,000 payments	~4,000 per year	Retained for 7 years for financial audit compliance
GPS Log	~1.5 million entries	~1.2 M per year	Retained for 6 months; older logs purged by scheduled job
Camping Gear	~10–20 items	Minimal	Retained indefinitely

8.4 Data Formats and Standards

- All timestamps are stored and transmitted in ISO 8601 UTC format (e.g., 2026-04-10T09:00:00Z).
- All monetary values are stored as DECIMAL(10,2) representing Sri Lankan Rupees (LKR) to two decimal places.
- GPS coordinates are stored as DECIMAL(9,6) following the WGS84 standard (World Geodetic System 1984).

- All API request and response payloads use the JSON data format with camelCase field naming conventions.
- All date-only fields (e.g., lastServiceDate) use the ISO 8601 date format: YYYY-MM-DD.
- Image files stored in AWS S3 shall be in JPEG or PNG format with a maximum file size of 5 MB per image.
- Exported reports are available in PDF and CSV formats.

8.5 Data Privacy and Compliance

- Personal data (name, email, phone, NIC number) is collected only with explicit user consent at the point of registration.
- NIC numbers and payment tokens are encrypted at the application layer using AES-256 before database storage.
- User passwords are never stored in plain text; BCrypt hashing with a cost factor of 12 is applied at all times.
- Users may request deletion or export of their personal data in accordance with applicable data protection regulations.
- Payment card data is never stored on the system; Stripe tokenization is used to ensure PCI DSS compliance.
- Audit log entries (user actions, admin activities, login events) are retained for a minimum of one year.

References

Table 29: References

[#]	Reference
[1]	IEEE, "IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions," IEEE, 2009.
[2]	IEEE, "IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications," IEEE, 1998.
[3]	ISO/IEC, "ISO/IEC 25010: System and Software Quality Models," ISO/IEC, 2011.
[4]	R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York: McGraw-Hill, 2014.
[5]	I. Sommerville, Software Engineering, 10th ed. Harlow: Pearson, 2016.
[6]	Spring Boot Documentation, "Spring Boot Reference Documentation," [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. [Accessed: Apr. 2026].
[7]	React Documentation, "React – A JavaScript library for building user interfaces," [Online]. Available: https://react.dev/. [Accessed: Apr. 2026].
[8]	PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," [Online]. Available: https://www.postgresql.org/docs/16/. [Accessed: Apr. 2026].
[9]	Google, "Google Maps Platform – Distance Matrix API," [Online]. Available: https://developers.google.com/maps/documentation/distance-matrix. [Accessed: Apr. 2026].
[10]	Stripe, "Stripe API Reference," [Online]. Available: https://stripe.com/docs/api. [Accessed: Apr. 2026].
[11]	Amazon Web Services, "AWS Well-Architected Framework," [Online]. Available: https://aws.amazon.com/architecture/well-architected/. [Accessed: Apr. 2026].
[12]	W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Apr. 2026].
[13]	OWASP, "OWASP Top Ten," [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 2026].